# *cs-mail* Product Build Plan

Server, Administration, Apple Member Application, and Sender Request Page

C-SQD

Draft 0.12 — 8 October 2026

**Abstract**

This plan turns the implemented Rust domain into a runnable product. It builds the core relationship-request loop end to end first—operator service, identity enrollment, request, decision, and settlement, driven from a command-line client—and then adds depth: a public request page for senders without an account, ordinary-email correspondence including member-initiated external conversations and review of their initial replies, the macOS member application, recovery across time and failure, the annual service and member-cycle experience, and open enrollment. cs-mail's core is also required by the future Phoros Account, so user testing informs design rather than deciding whether development continues. Open enrollment introduces no real redeemable value.

**Design principles.** Before changing the implementation or this design, read the Rust-domain design principles (`docs/rust-domain-principles.md`). They are binding for cs-mail and identity-model; new work must not regress them.

## 1 What Changed in Draft 0.12

The member application uses SwiftUI, with macOS shipping first and iPhone/iPad considered from the first slice. This replaces the planned Tauri shell. Business rules and portable client workflows remain in Rust. A narrow Rust-to-Swift interface and Apple library packaging move from Milestone 8 to Milestone 4. Apple targets share suitable views and presentation models; the guest request page keeps its own web components. Android and Windows later reuse the Rust client through separate interfaces and platform adapters. See the Apple client architecture. Hosting remains deferred until the macOS application is locally validated.

## 2 What Changed in Draft 0.11

Member-selected external conversations need no separate mailbox-control challenge. Automatic email authentication and exact mailbox routing remain required. Pending conversations allow unlimited uncharged replies; acceptance or an express lane establishes broader permission. Rejection closes only the unpaid invitation. Withdrawal of disclosure consent cancels waiting mail atomically; dispatch already begun may be too late to cancel. The first local Rust slice and CLI demonstration implement these rules with a development relay. A separate receive-only SMTP process now authenticates plain-text and paired plain-text/HTML replies, offers STARTTLS, and acknowledges only after durable admission. Its wire experiments use real DKIM verification with controlled DNS; public deployment is still work.

The outbound SMTP2GO adapter and authenticated feedback are locally validated. A fresh live email confirmed Gmail's acceptance of cs-mail's RSA-2048 signature and correct API receipt recording. Callbacks now have their own loopback listener and can run with sending disabled; public HTTPS/provider checks remain pending. API acceptance, delivery evidence and uncertain handoffs have separate meanings.

## 3 What Changed in Draft 0.7

- There are no invitations and no invite-only pilot. Enrollment is open to anyone who completes a fresh login and bank-ownership verification; the one-person-one-account rules are the only gate. Milestone 7 becomes an open-enrollment launch with simulated value.
- Request pricing is redesigned as Milestone 1b, ahead of Milestone 2: one cs-mail-wide operator policy (processing component $C$ and collateral bounds) and up to eight request classes per protocol identity, published independently of any relationship. See `docs/request-pricing-design.md`.

## 4 What Changed From Draft 0.4

Draft 0.4 built complete layers in sequence and reached the first relationship request only at its third and fourth milestones. This draft reorders the work:

- The core request loop runs end to end from a developer CLI at Milestone 2, before any graphical client.
- Enrollment and annual service use the existing identity-model and billing domains from the start, with simulated evidence supplied by development support code. No fixture or bypass path is later replaced.
- A public *sender request page* lets a person without a cs-mail account—a *guest*—pick a request class, pay, and write a message. After acceptance, guest correspondence is ordinary email in both directions, through the deployment's own mail receiver. The guest-sender design (Milestone 3a) is recorded in `docs/guest-sender-design.md`.
- The macOS application follows the proven loop. Billing and distribution *experience* moves later; its domain rules are already implemented.
- Each milestone names the conjectures its tests challenge, following principle 10 of the Rust-domain principles, plus a learning question that does not gate progress.
- Development and testing are local. There is no hosted CI.

## 5 Build on the Implemented Domain Contracts

The *Protocol Specification* governs requests and permission. The normative C-SQD financial-program section of the *Deployment and Migration Profile* governs annual utility billing, the pool, and equal member distributions. The *Rust Reference Architecture* describes the current ownership boundaries. Start with the C-SQD domain model for confirmed rules and deferred choices, the product identity model for accounts and logins, and the Rust-domain principles for how new code is shaped and validated.

The workspace already implements the request lifecycle with recipient-defined request classes, lane acceptance with atomic refund settlement, native correspondence, consent-scoped decryption, request-specific payment operations and refunds, forfeiture maturity, annual allocation, fixed service contracts with advance collection, funded coverage, and automatic lump-sum distribution preparation. Signed service and client APIs, PostgreSQL transactions, and worker libraries exercise those rules with a simulated external provider. The local server, CLIs and guest request page are runnable; the Apple member application remains to be built.

Product work reuses those owners and APIs. It does not recreate deprecated labels, aliases, fallback readers, or parallel implementations. When product work shows that a domain rule is wrong or incomplete, the change is made as an explicit, reviewed cutover in the owning domain, not patched in an adapter.

# 6 The Product and Its Users

**Clients and their roles.**

- **Member application (SwiftUI; macOS first).** Holds endpoint keys, the encrypted inbox, request decisions, request classes, lanes, blocks, and financial status. Native content is end-to-end encrypted.
- **Sender request page (web).** Lets a guest—a person without a cs-mail account—select a recipient's published request class, authorize one conditional charge, and write a message. The page is served by the provider, so it cannot carry the end-to-end promise; its messages are labeled as an explicit privacy downgrade.
- **External email conversations.** A guest may approach through a paid request, or a member may send to an external mailbox first and review authenticated replies without a charge within the pending conversation. Broader access requires acceptance or an express lane. The member's emails arrive from the member's cs-mail address; the guest's replies return through the deployment's own mail receiver. The member consents to the downgrade per conversation.
- **Developer CLI.** Drives member journeys before the desktop application exists and remains a development and operations tool.
- **Administration CLI.** An authenticated operator client.

A full web inbox for members is not planned. It would place endpoint keys in code delivered by the provider on each page load, which conflicts with the promise that the provider cannot decrypt content. The client core compiles to WebAssembly, so the option remains available if that trust model is ever accepted or mitigated.

**Enrollment.** Open to anyone who completes a fresh login and bank-ownership verification. There are no invitations. The one-person-one-account rules are the only gate: a cs-mail login belongs to one account, and an enrollment whose evidence (legal name, date of birth, bank account, address, phone) shares its bank account, or any two fields, with another is held until a CSQD operator approves or denies it. Accounts approved this way may share a bank account. The service is run by CSQD, the only provider, in one region on one PostgreSQL deployment. The intended recipients are people who receive substantial unsolicited contact; senders are people trying to reach them, including guests once the request page ships.

**Value.** A deterministic external-payment simulator approves, declines, delays, or reports an uncertain charge; confirms or fails a refund; and executes or fails a member payment. All amounts are explicitly nonredeemable. Users receive no spendable test-credit wallet; the operator configures simulator scenarios.

**Out of scope for this plan.** Independent-provider federation, general mailbox hosting (such as IMAP access or hosting external accounts), live payments, mobile releases, and multi-region operation. Member-initiated external conversations, guest-initiated conversations, and transactional notices are in scope.

# 7 Architecture of the Product Layer

```
crates/
  existing cs-mail-* crates     # domain, application, storage, workers
  cs-mail-client                # extended client core (ports for keys,
                                #   local storage, transport, clock)
  cs-mail-api-types             # add when transport contracts need it
  cs-mail-client-ffi            # narrow client facade; Swift in Milestone 4
apps/
  cs-maild                      # HTTP API, workers, sender page hosting,
                                #   outbound guest and notice email
  cs-mail-mx                    # own receive-only SMTP receiver (MX),
                                #   using cs-mail-smtp-gateway
  cs-mail-admin                 # authenticated operator CLI
  cs-mail-cli                   # developer member client
  cs-mail-apple                 # SwiftUI macOS/iOS targets, shared views,
                                #   Apple adapters and Rust library package
  cs-mail-web                   # sender request page (static assets)
../identity-model               # separate repository, sibling checkout
```

Dependencies point inward: applications use the client core and application layer, which use the domain; domain crates never depend on applications. **Applications are adapters.** They validate input, invoke application use cases, and render outcomes. They make no domain decisions, obtain trusted time only as the application-persistence boundary prescribes, and never translate a signed command into a different meaning. The Apple application calls the native Rust client through a narrow binding interface. SwiftUI owns presentation; Rust owns business rules and portable client workflows. Apple platform adapters supply Keychain, local persistence and lifecycle integration through client ports. The Swift bindings and device/simulator packaging begin in Milestone 4; these planned paths are introduced with their first consumer. WebAssembly is not required for the Apple application. The guest page keeps its own web components, sharing terminology and API contracts rather than rendered UI with SwiftUI.

identity-model remains a separate repository because it will also serve the Phoros Account. cs-mail pins the reviewed identity sources in `identity-source.sha256`; commit identity-model changes before the cs-mail changes that depend on them and keep the manifest in step.

## 8 Contracts That Apply to Every Milestone

**Keys and authority.** Endpoint private keys use operating-system secure storage (or, for the developer CLI only, a local-file implementation of the same key-custody port) and never enter logs, configuration, SQLite, provider APIs, or diagnostics. Device enrollment and revocation require authenticated authority. Administrative credentials are distinct from user credentials; the admin CLI uses an authenticated API, never direct table writes.

**Evidence and simulation.** Simulated login, bank-ownership, funding, and payment evidence, together with scenario builders, live in development support code. They enter the product through the same ports as real evidence and are never presented as business commitments. Rates, thresholds, prices, and dates use explicitly versioned simulation policies.

**Outcomes and uncertainty.** Every visible outcome has a durable cause and a stable retry identity. "Outcome unknown," "failed," "expired," and "denied" remain distinct through storage, API, and interface. The interface separates submission preparation, awaiting a decision, refund due, refund completed, annual service coverage, finalized member allocation, and payment completion. A distribution never marks a service year unpaid. No screen suggests that rejecting a sender pays the recipient or that a payable can fund a request.

**Versions and data.** Protocol, wire, API, event, policy, and database versions are separate. Until the project owner declares migrations forward-only, schema changes may replace earlier migrations; afterwards migrations are forward-only. Logs contain scoped references and typed errors, not plaintext, secrets, identity mappings, full signed commands, or complete financial snapshots. Retention policy is explicit before user data exists.

**Validation.** Tests challenge named conjectures whose falsification would matter; each milestone lists its claims and agrees its testing scope before work begins. Claims about locking, atomicity, or concurrency are challenged against PostgreSQL, not an in-memory adapter. Guarantees already enforced by types need no redundant runtime test. Demonstration scripts show a journey; they are not evidence for a claim. At each handoff, run the local quality gate, which reports the identity source manifest, dependency direction, rustfmt, Clippy, workspace tests, PostgreSQL suites, and application binaries separately:

```
CS_MAIL_TEST_DATABASE_URL=... scripts/quality-gate.sh
```

## 9 Milestone Map

| Stage | Result | Exit evidence |
|---|---|---|
| 0 | Product skeleton | Buildable apps, inward dependencies, quality gate |
| 1 | Service and identity boot | Clean start, restart, open enrollment, simulator scenarios |
| 1b | Request pricing | Operator policy with collateral bounds; per-address classes |
| 2 | Core loop from the CLI | Every request outcome, from a clean database |
| 3a | Guest-sender domain design | Recorded design; open decisions settled |
| 3b | Sender request page | A guest completes a request; every outcome settles |
| 3c | External email correspondence | Both entry paths and initial-reply decisions |
| 4 | SwiftUI Apple application, macOS first | Milestone 2 journeys, Keychain, early iOS checks |
| 5 | Time and failure | Crash, race, and deadline claims challenged |
| 6 | Annual service and member cycle | A full simulated year, including nonrenewal |
| 7 | Open enrollment | Signed builds, recovery, restore, launch findings |
| 8 | Portability and Phoros | Android/Windows adapters, client parity, Phoros integration |

Each stage depends on the preceding one, except that Milestone 4 may proceed before the public-deployment portion of Milestone 3c. A milestone is complete when its demonstration runs from a clean checkout and its claims have been challenged, not merely when new code compiles.

**Current sequence, agreed 8 October 2026.** Build and validate the SwiftUI macOS member application locally before provisioning or paying for a rented Linux host. Milestone 4 is the next implementation priority, with macOS and iOS targets from the first slice and no committed iOS release date: onboarding, inbox, request decisions, conversations, disclosure controls, and quit/reopen and failure recovery against the local Rust service and simulated payments. The locally implemented email adapters and controlled transport fixtures support this work. Public MX, HTTPS and actual SMTP2GO callback validation wait until the application is ready for a hosted external-email pilot. A rented Linux server remains the hosting direction; Amazon Lightsail is a candidate, not a purchase commitment. Retain the prepared deployment package for that later step.

## 10 Milestone 0: Product Skeleton

Create `apps/cs-maild`, `apps/cs-mail-admin`, and `apps/cs-mail-cli` with shared package metadata, lints, and dependency versions. Each reports its version and help. Add a local script that runs the full quality gate without hiding individual checks, and document the inward dependency rule.

**Migration policy:** until the project owner declares migrations forward-only, a schema change may replace or renumber existing migrations and requires a fresh database. From that declaration, migrations in cs-mail and identity-model are forward-only: a merged migration is never edited, removed, or renumbered.

**Exit:** clean checkout, passing gate, running binaries. No listener, enrollment, or domain change belongs to this stage.

**Status: complete (25 September 2026).** The three binaries report their version and help. `scripts/quality-gate.sh` runs the identity source manifest, dependency direction, rustfmt, Clippy, workspace tests, PostgreSQL suites, and binary checks, each reported separately; see `apps/README.md` for the boundary rules.

## 11 Milestone 1: The Service and Identity Boot

The operator can migrate an empty database, bootstrap CSQD as the provider idempotently, start the API, publish service offers, and select simulator scenarios—without editing SQL. Deployment-level state (configuration, accounts, billing, and, from Milestone 1b, request pricing) is reached through a deployment-scoped storage handle, not a relationship-scoped engine. Use Tokio, Axum, Rustls, structured tracing, and Clap. Keep synchronous storage behind a bounded blocking boundary until a tested replacement is warranted. Readiness distinguishes process health from database readiness.

The identity-model enrollment service runs beside `cs-maild`, with a development OIDC provider (the Keycloak realm in `identity-model/ci`) and simulated bank-ownership evidence from development support code. Enrollment is open: a fresh login and bank-ownership verification are the only requirements, and the one-person-one-account rules are the only gate. The identity service screens each applicant's evidence under cs-mail's policy and holds overlaps for operator review (`DOMAIN_MODEL.md`). The payment simulator persists operation identities and supports delayed, duplicate, out-of-order, and failed results; its scenario changes are audited and separate from canonical financial writes.

**Claim (PostgreSQL).**
Repeating bootstrap cannot install a second provider signing key or replace the key registry.
*Falsified by:* Two bootstrap runs, including one interrupted midway, leave two keys accepted as CSQD's or a changed registry.
*Matters because:* Two keys could then sign quotes and receipts as the provider.

**Claim (PostgreSQL).**
An enrollment whose evidence shares a bank account, or any two of name, date of birth, bank account, address and phone, with another enrollment cannot activate until a CSQD operator approves it, including when both enroll concurrently; a denial is final.
*Falsified by:* Two overlapping enrollments both become eligible without review, or a held or denied enrollment activates.
*Matters because:* One account per person is an anti-abuse objective and a member-share rule.

**Claim (PostgreSQL).**
A restart cannot duplicate a simulated payment operation.
*Falsified by:* A killed and restarted service shows two operations for one operation identity.
*Matters because:* Duplicate operations become duplicate charges once payments are real.

**Exit:** a clean-start transcript, restart demonstration, and redacted diagnostics.

**Status: implemented (25 September 2026)**; the Keycloak enrollment run remains. `cs-maild` migrates, bootstraps (`migrate`, `bootstrap`) and serves a loopback HTTP API with process health (`/health`) separate from database and bootstrap readiness (`/ready`). Operator commands (service offers, request pricing, and, in development builds, simulator scenarios) are signed with an operator key named in the bootstrap record and replay their recorded outcome when repeated; `cs-mail-admin` generates keys, signs, and sends them. Open enrollment is served at `/v1/enrollments`. An enrollment whose evidence overlaps another's is held until an operator approves or denies it with `cs-mail-admin review`; accounts approved this way may share a bank account. The payment simulator is a durable `PaymentProcessor` with its own schema and audited scenario commands. `scripts/m1-demo.sh` produces the clean-start, restart, and redaction transcript from an empty database; each claim above has a PostgreSQL test.

Decisions and deviations, for confirmation:

- TLS terminates at the deployment proxy; `cs-maild` listens on loopback only and does not embed Rustls.
- A lost database connection is not reopened in place: the service exits with a failure status for its supervisor to restart it. Connection pooling is Milestone 5 work.
- Enrollment identifiers derive from the client's operation identity, so a retried request replays rather than reserving a second account.

Not yet covered: enrollment against the Keycloak realm (it was exercised with a development token verifier in its place); rotating identity trust and configuring the financial program remain outside the operator API; account changes (device recovery, bank rebinding) do not screen evidence, and rebinding to a bank account another account enrolled with is refused; phones are not yet verified by a code; normalization does not equate address abbreviations ("Street" and "St").

## 12 Milestone 1b: Request Pricing

Replace the per-recipient pricing menu and relationship-scoped class publication with the agreed design in `docs/request-pricing-design.md`. CSQD publishes one versioned operator policy—processing component $C$ and cs-mail-wide collateral bounds—through the authenticated administration API. Each protocol identity publishes up to eight request classes, each with collateral within the bounds, signed by that address's own key and independent of any relationship. A class outside the current bounds cannot be quoted until the recipient republishes; issued quotes keep their terms. Quote prices become their own value rather than fields of the policy snapshot. The replaced tables, APIs and menu types are removed without aliases.

**Claim (PostgreSQL).**
A class outside the current bounds cannot be quoted, although its publication was valid when made.
*Falsified by:* After the operator narrows the bounds, a quote is issued for a class outside them.
*Matters because:* Bounds that do not bind would let old classes escape policy indefinitely.

**Claim (PostgreSQL).**
A publication signed for one address cannot be applied to another, including another address of the same account.
*Falsified by:* A publication signed with address A's key changes address B's classes.
*Matters because:* Each address's classes are its own consent to paid approaches.

**Claim (PostgreSQL).**
A quote issued concurrently with a republication or policy change carries the terms of exactly one publication and one policy version.
*Falsified by:* An issued quote combines $C$, $S$, or a description from different versions.
*Matters because:* The charge and refund obligation must match what the sender was shown.

**Exit:** the operator changes the policy and a recipient's out-of-bounds class stops being quotable until republished, from a clean database.

**Status: implemented (25 September 2026)**, with the deployment-scoped storage handle; the three claims are challenged against PostgreSQL. The operator policy is published through the authenticated administration API (Milestone 1).

## 13 Milestone 2: The Core Loop From the Command Line

Alice and Bob enroll through identity-model using the developer CLI and establish their protocol identities and device keys. Each purchases annual service through the existing billing domain; the simulator finalizes funding and coverage begins within the purchased interval.

Bob publishes request classes. Alice resolves Bob, reviews $C$, $S$, the total conditional charge, the decision deadline, and possible refunds; authorizes one charge; and submits an encrypted initial message after funding reaches finality. Bob accepts with standing permission or an express lane, rejects, blocks, or lets the request expire. Alice sends follow-ups when allowed. Both sides see settlement: refund owed and confirmed, processing revenue retained, collateral pending forfeiture. Uncertain outcomes are shown as uncertain.

**Claim (PostgreSQL).**
Retrying a submission whose outcome is unknown cannot create a second request or charge.
*Falsified by:* A client that loses the response and retries produces two requests or two captures.
*Matters because:* One request must remain one charge.

**Claim (PostgreSQL).**
Acceptance commits permission and the full refund obligation together or not at all.
*Falsified by:* A failure injected during acceptance leaves a lane or permission without its refund obligation, or the reverse.
*Matters because:* Either half alone breaks the settlement contract.

**Claim (in-memory and PostgreSQL).**
An altered retry under an existing operation identity is rejected.
*Falsified by:* A retry with changed terms or content under the same identity is accepted.
*Matters because:* Retry identities would stop protecting request terms.

**Exit:** a scripted two-member demonstration of every outcome from a clean database. **Learning:** does the decision flow read clearly, and which wording confuses?

**Status: implemented (26 September 2026).** `scripts/m2-demo.sh` runs the loop from clean databases: members enroll through the identity-model enrollment service, buy annual service, publish request classes and content keys, and send requests that are accepted with standing permission (with a follow-up and a lost response retried on the way), accepted with an express lane, rejected, left to expire, cancelled before submission, and blocked then unblocked. Both sides see the charge, refunds owed and confirmed, processing revenue retained and collateral pending forfeiture; operations the processor has not confirmed are shown as uncertain. Bob reads and decrypts his messages from the CLI. The three claims above are challenged against PostgreSQL through the member gateway, and the third also in memory.

The loop consists of:

- a signed operator command, `PublishProtocolPolicy`, for the request windows, cooldowns, backoff and member-program terms; the bootstrap record fixes the rest of the policy and names a scheduler key for the service's own deadline commands;
- relationships opened by the service for each (sender, recipient) pair, with references derived by the privacy deriver under host secrets (the request history from the sender's private principal), and provider and scheduler keys installed when a relationship is first opened;
- a member API in `cs-maild`: signed relationship commands, account and billing commands, request classes, endpoint content-key publications, encrypted content with its sender certificate, express-lane grants, and signed, fresh member reads that return only the reader's own account and relationships;

- a relationship worker for received commands, artifact signing, deadlines, request payments and delivery to the stored mailbox;
- `cs-mail-cli`, which journals every signed request before sending it and resends the same bytes after a lost response.

Decisions and deviations, for confirmation:

- Enrollment in the demonstration uses a development login (`cs-mail-test-support` example `dev-identity`): the real identity application and API with an unsigned development token in place of an OIDC ID token. The identity service cannot trust a local Keycloak's self-signed certificate, so the Keycloak run remains open (Milestone 1).
- A member adds its recipient role by registering a second key through the existing `RegisterKey` account command; keys stay bound to one role.
- An express lane's native subject is the relationship's own reference.
- Delivery within the deployment completes when the ciphertext is stored; members read it through authenticated reads.

Not yet covered: relationships are created for any two enrolled addresses before the command is authenticated (the relationship holds no state until an authenticated command succeeds); member reads are fresh and signed but not recorded as used, so a captured read can be replayed within its five-minute window; the relationship worker runs only with the payment simulator; and accepted members cannot yet send ordinary messages outside a request from the CLI.

## 14 Milestone 3a: Guest-Sender Domain Design

The current domain recognized two senders: members, with an account, verified bank association, and protocol identity; and organizational legacy senders, identified by a DMARC-authenticated domain. A guest is neither. The agreed design is recorded in `docs/guest-sender-design.md` and summarized in the domain model:

- A guest is a distinct sender kind, identified by one verified mailbox and mapped at the edge to a synthetic protocol identity, following the legacy-sender precedent. The request kernel is unchanged; lanes gain a guest-mailbox subject.
- Mailbox verification establishes control of the mailbox, not personhood. The payment processor's instrument fingerprint is an anti-abuse signal, not an identity claim.
- Guest request history is grouped by both mailbox and payment fingerprint; linked histories merge prospectively to the most restrictive state.
- Accepted guests communicate free and indefinitely, subject to the recipient's block and revocation. Refunds return to the funding payment method; forfeitures enter the member pool.
- The recipient sees the guest's email address. Guest content is not end-to-end; each member reply by email requires per-conversation downgrade consent.

**Decided (25 September 2026):** a receive-only Rust receiver (`apps/cs-mail-mx`); SMTP-level rejection of a blocked guest's mail; outbound email through a delivery service at first, DKIM-signed with the deployment domain's key; and indefinite retention of mailbox and payment fingerprint evidence by default, disclosed to the guest before payment. **Exit:** complete; the domain model reflects the design.

## 15 Milestone 3b: Sender Request Page

Each member has a public request link. A guest verifies their mailbox, selects a published class, reviews $C$, $S$, the refund rules, the privacy label, the fact that the recipient will see their address, and the indefinite retention of their mailbox and payment evidence, authorizes one simulated charge, and writes a message. Encryption to the recipient's key on arrival is an application use case; plaintext is not stored or logged. The recipient decides from the developer CLI until Milestone 4. Verification codes and outcome and refund notices are sent as DKIM-signed email from the deployment domain.

**Claim (PostgreSQL).**
Changing mailbox while reusing a payment instrument cannot reset request history or bypass a pending request.
*Falsified by:* A second mailbox with the same fingerprint submits while the first request is pending, or at a lower history level.
*Matters because:* The request page would become a harassment channel.

**Claim (PostgreSQL).**
No two distinct sender subjects resolve to the same protocol identity.
*Falsified by:* Two subjects of any kind are registered under one identity.
*Matters because:* Permission or blocks would apply to the wrong sender.

**Claim (inspection and integration).**
Guest-message plaintext is never written to storage, logs, or diagnostics.
*Falsified by:* A secret-scan of the database, logs, and diagnostics after a guest request finds the message text.
*Matters because:* The downgrade label would understate the exposure.

**Claim (PostgreSQL).**
A guest refund returns only to the payment method that funded it.
*Falsified by:* A refund is directed to any other destination.
*Matters because:* Refund redirection is a fraud path.

**Exit:** a guest completes a request end to end and every outcome settles. **Learning:** do strangers understand the page, and what collateral feels reasonable?

**Status: implemented (26 September 2026).** `scripts/m3b-demo.sh` runs from clean databases: Bob enrolls and publishes request classes, and guests use his request page in headless Chromium. Requests are accepted, rejected, cancelled after the charge and before sending, and left to expire; a declined card makes no request; a second address with the same card is held by the first address's history. Bob decides from the CLI and sees each guest by the verified address. Verification codes and outcome and refund notices are DKIM-signed and written to a development outbox. After a restart, a scan finds no guest message text, card number, verification code, secret or database URL in the database or the log. The four claims above are challenged against PostgreSQL through the guest gateway; the log half of the third is covered by the demonstration.

The request page consists of:

- `cs-mail-guests`: guest mailboxes and their synthetic protocol identities, the sender-subject kinds, mailbox verification and sessions, the history rules over mailbox and fingerprint evidence, the commands a guest may issue, and the notices owed to guests;
- a sender-subject registry in storage: every protocol identity has exactly one subject (member, legacy domain or guest), registered with enrollment, persona addition, legacy grants and a guest's first request;

- opening a guest relationship as one transaction: the subject, the history created or merged with the evidence that caused it, the evidence links, the card token as a funding source, and the relationship;
- guest commands through the unchanged request kernel, as a trusted ingress authenticated by the verified session; the message is encrypted to the recipient on arrival under a provider guest content key, and declared human-initiated through the guest gateway;
- `cs-mail-web`, a static page served by `cs-maild` at `/r/{persona}` with a JSON guest API, and a notices worker.

Decisions and deviations, for confirmation:

- "Pending" in the first claim is the history's reservation, as the guest-sender design defines it. Once a request is submitted the reservation is released and the history's backoff applies instead, so a second address with the same card may ask again after the backoff, one level up, while the first request awaits its decision. Members' aliases behave the same way. Holding the history until the decision would be a request-kernel change.
- Evidence that would merge two histories each holding a reservation is refused rather than merged.
- The guest authorizes the charge, then sends or cancels as a separate step, so cancellation before sending is a real choice rather than a race.
- A recipient sees only submitted requests and admitted messages: a request cancelled before sending, and its content, never reach the recipient. This applies to members as well.
- A guest instrument is a funding source of its own kind; a restriction on it is permanent, since nothing re-verifies it.
- A guest's answer carries the protocol outcome without a provider receipt: the guest holds no key to verify one. The receipt is still signed and kept.
- Express lanes for guests are deferred to Milestone 3c.

Not yet covered: a real payment processor and email delivery service (the simulator tokenizes cards and the outbox is a directory); the page has no bot protection beyond per-mailbox code limits; opening a relationship commits before the protocol decides, so a refused guest still leaves a subject, evidence links and an empty relationship (not shown to the recipient); legacy-domain collisions are exercised only by construction, not by a test; and the member request path may report a missing receipt when the relationship worker is signing it at the same moment.

## 16 Milestone 3c: External Email Correspondence

Support both entry paths: a guest approaches through the request page, and a member starts a conversation by entering an external address. The member-selected path requires no mailbox-control challenge. Automatic incoming authentication and exact mailbox matching remain required. Unlimited uncharged replies are confined to the pending conversation until rejection or block. Accept and express lanes establish broader directional access using the existing relationship owner. Rejection closes the unpaid invitation without changing paid-request history or creating settlement; a later paid approach remains possible unless blocked.

Outbound disclosure consent is conversation scoped. Withdrawing it cancels waiting messages atomically; after dispatch begins cancellation may be too late. Unknown handoffs require reconciliation and cannot be automatically redispatched.

The first Rust slice provides signed member commands, encrypted authored-text copies, pending-reply admission at the trusted receiver boundary, and development relay handoff. The reproducible CLI/HTTP journey is `scripts/m3c-demo.sh`. PostgreSQL experiments challenge

admission scope, permission changes, transaction failure, retries, and withdrawal versus dispatch. The receive-only `cs-mail-mx` adapter now adds real automatic authentication, STARTTLS, known-conversation routing and durable SMTP retries. It accepts plain text or a flat plain-text/HTML alternative, selecting text and authenticating the original email. Four wire experiments challenge forged senders, current permission, atomic commit, retries and unsupported MIME, and join an outgoing HTTP fixture to a readable incoming reply. They use local DNS and PostgreSQL. See `docs/smtp-receiver.md` for operating instructions and test limits. The SMTP2GO HTTPS MIME adapter now submits DKIM-signed threaded plain-text mail after committed dispatch. Authenticated provider events reconcile uncertain handoffs atomically and remain separate from acceptance; late events cannot erase earlier delivery evidence. Local HTTP/PostgreSQL experiments challenge cancellation, duplicate submission and feedback binding. External correspondence no longer requires the guest payment simulator. See `docs/smtp2go-outbound.md`. A fresh live test confirmed Gmail's acceptance of cs-mail's RSA-2048 signature and correct fast-accept receipt recording. Callbacks now use a dedicated loopback HTTP listener with sending independently disabled. An HTTP/PostgreSQL experiment challenges route isolation, authentication, acknowledgement after commit and lost-response retries. Public callback HTTPS/provider checks, MX/MTA-STS, HTML-only mail and attachments, general member-address mail and native-source exports remain subsequent work.

Complete and deploy `apps/cs-mail-mx`, the deployment's own receive-only SMTP receiver and MX for the deployment domain. It is a separate, minimally privileged process that hands each message with its full connection context to `cs-mail-smtp-gateway`, answers `250` only after durable admission, uses permanent `5xx` for refusals and `4xx` for temporary failures, offers STARTTLS, and publishes MTA-STS. The member's first message and subsequent emails are sent as threaded email from the member's protocol identity address, with a per-conversation reply address, under that conversation's downgrade consent. Every inbound message requires automatic email authentication and exact From-mailbox matching, including replies to a member-initiated pending conversation. DMARC must pass and align with the member-selected or verified mailbox; mail from domains with missing or permissive DMARC policy is held for confirmation on the request page. Unknown senders to a member's address receive an automatic reply with the request link only when DMARC passes; otherwise the message is rejected, avoiding backscatter. A blocked or revoked guest's mail is rejected at the SMTP level. Send outbound mail through an email delivery service, DKIM-signed with the deployment domain's key; the downgrade consent discloses that the service sees reply plaintext. Publish SPF, DKIM, and DMARC for the deployment domain; process bounces and complaints.

**Claim (integration).**
A message with a forged From address for a guest's mailbox is not admitted.
*Falsified by:* A message failing DMARC alignment for the guest's domain reaches the member.
*Matters because:* Anyone could speak as an accepted guest.

**Claim (integration).**
A forwarded reply token alone cannot post into the conversation.
*Falsified by:* A message from a different mailbox carrying a valid token is admitted.
*Matters because:* Forwarding a thread would grant permission.

**Claim (PostgreSQL).**
No member reply leaves as email without that conversation's downgrade consent.
*Falsified by:* An outbound message is sent for a conversation lacking consent.
*Matters because:* The member's plaintext would leave without agreement.

**Claim (integration).**
A blocked or revoked guest's mail is rejected with a permanent SMTP error and never delivered.
*Falsified by:* Mail from the guest's mailbox is accepted with `250` after the block or revocation.
*Matters because:* Blocking is the recipient's primary control once guests message freely.

**Claim (PostgreSQL, crash injection).**
The receiver never answers `250` before the message is durably admitted.
*Falsified by:* A crash after `250` loses the message, or an accepted message is later refused.
*Matters because:* Senders would believe mail was delivered when it was not.

**Exit:** from a clean deployment, demonstrate both a paid guest request followed by threaded correspondence and a member-initiated email followed by an uncharged replies. Show the conversation pending review, each permission decision (standing acceptance, express lane, rejection), and subsequent access according to the decision and the recorded policies. Authentication alone and the first outbound send must not create broader standing permission. **Learning:** does correspondence feel like ordinary email to guests, and is the member's initial-reply decision clear?

## 17 Milestone 4: SwiftUI Apple Application, macOS First

Alice installs the application, enrolls or recovers through identity-model, and reopens the same account after quitting. Keys live in the macOS keychain behind the key-custody port; SQLite holds ciphertext, key references, cursors, minimized relationship metadata, and nonsecret preferences only. Portable client use cases coordinate enrollment and device management; the SwiftUI application stays thin. Introduce the Rust client facade, Swift bindings and Apple library packaging here. Define typed results, asynchronous calls, cancellation and ownership explicitly; UI cancellation does not cancel a command already accepted by the server. Start with macOS and iOS targets, share appropriate presentation models and views, and adapt navigation and lifecycle behavior to each platform. macOS ships first. The first slice begins with browser sign-in and first-device enrollment through the existing identity-model integration. It then reopens the account, synchronizes and reads one request, and accepts or rejects it through the existing service. Check this slice on iPhone early before expanding to the full milestone journeys. See the Apple client architecture.

Bob sees the chosen class, one request with its grouped messages, and a clear deadline, and decides with permission explained first and money second. He manages each address's request classes with collateral within the operator's bounds, follow-up preferences, lanes, and blocks. Financial status shows request charges and refunds separately from annual service and distributions. Apple interface components are shared where appropriate across macOS and iOS. The guest request page shares terminology and design intent, with its own web implementation.

The member can also compose to an external email address with explicit downgrade consent, read authenticated replies in a pending conversation, and choose Accept, Reject, or Grant an express lane. These uncharged replies are visually distinct from a paid guest request. This uses Milestone 3c's local email implementation and controlled transport fixtures; public hosting and live external-email validation follow the locally validated macOS application.

**Claim (integration).**
Secret key material never reaches SQLite, logs, or diagnostics.
*Falsified by:* A secret-scan after enrollment, messaging, and export finds key bytes.
*Matters because:* The endpoint-encryption promise would be void.

**Claim (integration).**
After quit, reopen, and projection rebuild, every displayed outcome matches the server's canonical record.
*Falsified by:* A decision, refund, or uncertain charge appears differently after reopening.
*Matters because:* Users would act on false financial or permission state.

**Exit:** Milestone 2's journeys in the application, including keychain denial and quit/reopen. Record early iOS layout and lifecycle checks separately from macOS release evidence; an iOS build alone does not establish device behavior. Agree the experiment scope before adding client tests. **Learning:** do recipients publish classes, how many, and how long do decisions take?

## 18 Milestone 5: Recover Across Time and Failure

Run scheduled deadlines, outbox delivery, payment reconciliation, and retention through supervised leased workers with a distinct scheduler authority, bounded retries, graceful shutdown, database TLS, and a bounded connection pool. Jobs recheck current state and use stable operation identities. Offline recipients and cursor replay must not duplicate delivery or financial effects.

**Claim (PostgreSQL, crash injection).**
A crash between capture and recording its response cannot lose or duplicate the charge.
*Falsified by:* After restart, reconciliation shows no capture or two captures for one request.
*Matters because:* Money would be lost or taken twice.

**Claim (PostgreSQL, race).**
Cancellation racing a delayed funding confirmation returns captured money exactly once.
*Falsified by:* The request is both cancelled and submitted, or the refund is issued twice or never.
*Matters because:* Cancellation is the user's safety valve before submission.

**Claim (PostgreSQL).**
Out-of-order provider events cannot reopen a terminal settlement.
*Falsified by:* A late event changes a settled request's obligations.
*Matters because:* Terminal financial outcomes must stay fixed.

**Claim (PostgreSQL).**
Coverage expiring while a request is unsettled stops new communication but not decision or settlement.
*Falsified by:* The recipient cannot decide, or a refund stalls, after coverage ends.
*Matters because:* Obligations outlive service coverage.

**Decision to record:** the bounded-acceptance timing edge cases currently deferred in the domain model.

## 19 Milestone 6: Annual Service and Member-Cycle Experience

Build the billing and distribution experience and run collection retries in operation. A member reviews a published offer with annual price, advance collection date, and fixed service interval; failed and late collection retries preserve the price and interval; a pending charge does not imply renewed service.

Finalize a simulated UTC calendar year from fixed funding and eligibility inputs, showing the once-only corporate assessment, equal minor-unit allocations, and restricted remainder. Pay each member one lump sum of the previous year's share to the associated verified bank account. With $U = \$120$, allocation scenarios of \$80, \$120, and \$170 each produce one payment, reported as rebate/excess of \$80/\$0, \$120/\$0, and \$120/\$50. Repeat with nonrenewal and closure, preserving the minimal payment route.

**Claim (PostgreSQL).**
A failed or unanswered payout attempt cannot create a second allocation or payable.
*Falsified by:* After failure and restart, a member has two payables for one pool year.
*Matters because:* Pool funds would be distributed twice.

**Claim (PostgreSQL).**
A later allocation year cannot reassess or re-divide amounts already assigned.
*Falsified by:* Year $n + 1$ changes an assigned year-$n$ amount.
*Matters because:* Members' fixed shares would be unstable.

**Claim (integration).**
A distribution never marks a paid service year unpaid or revokes coverage.
*Falsified by:* Coverage or charge status changes when a distribution is paid.
*Matters because:* Service and distribution are separate transactions by rule.

**Exit:** a full simulated year, including a nonrenewing member who is still paid.

## 20 Milestone 7: Open Enrollment With Simulated Value

Package and sign the macOS application and versioned server and CLI artifacts. Complete user-held recovery and trusted-device enrollment with explicit device revocation and no provider decryption authority. Provide onboarding for annual service, requests, refunds, simulated distributions, follow-up preferences, encryption, recovery, and device loss; scoped diagnostics export; support and abuse intake; deletion procedures; backup and restore; and release rollback. Backups and retained obligation records respect separate keys and access roles.

Open enrollment to the public, aiming first at recipients who receive substantial unsolicited contact and senders who want to reach them, including guests if Milestones 3b and 3c have shipped. Before live payments, confirm that the chosen bank integration reports each account owner's name, address and phone, and decide whether phones are also verified by a text-message code. Configure the deployment domain's MX, SPF, DKIM, and DMARC records and monitor outbound deliverability. Nondeveloper testers use clean machines and the user guide without direct database repair.

**Claim (integration).**
Restore followed by reconciliation reproduces every contract, request, refund, allocation, payable, and inbox state.
*Falsified by:* Any obligation differs between the pre-backup and reconciled post-restore state.
*Matters because:* Restore is the operator's last line of defense.

**Claim (integration).**
A revoked device cannot authorize commands or decrypt new content.
*Falsified by:* A command signed by a revoked device is accepted, or new content is readable to it.
*Matters because:* Device loss is a routine event.

**Exit:** signed releases, restore and recovery records, resolved critical usability findings, and a written summary of launch findings.

## 21 Milestone 8: Portability and Phoros

Extend the client facade introduced for Swift in Milestone 4 to Android and Windows once the Apple path is stable. Reuse Rust use cases for identity, requests, decisions, encrypted inbox, billing and coverage, financial status, distributions and devices. Supply native interfaces and platform adapters, preserving bounded values, stable errors, cancellation and explicit ownership. Kotlin/Compose and C#/WinUI are candidate interfaces; validate binding and packaging choices before selecting them. Run the same consequential client stories through each adapter, including device-specific secure storage and lifecycle behavior. iOS already shares the Apple application structure; its release readiness requires separate validation and scheduling. Add a live payment connector.

For Phoros, its enrollment ceremony adopts an existing cs-mail subject through verified account control, as the product identity model prescribes. Version the identity contract as a published interface once two products consume it. Design cross-product account linking on explicit, granular, revocable consent.

## 22 Deferred and Open

- Bank-account replacement, full account management, and detailed erasure policy.
- Independent-provider federation and general mailbox hosting (such as IMAP).
- Converting a guest into a member while keeping existing relationships.
- A full web inbox for members.
- External email: live transport integration and native-source exports.

## 23 How an Implementation Agent Should Proceed

For each slice, read the applicable contracts, the Rust-domain principles, and local instructions. Name one user-visible outcome, identify its trust and persistence boundaries, agree the testing scope, then implement contracts, application orchestration, adapters, and interface in that order. Report what changed, what was demonstrated, which claims were challenged, and what remains open or untested.

Do not bypass signed execution, write canonical tables from a CLI, persist plaintext for convenience, hide an uncertain charge, add tests merely to accompany code, or weaken an existing check to finish a milestone. A domain change requires an explicit decision and a reviewed cutover. Scope does not expand to live value, federation, or general email hosting because an adapter exists.

If reconciliation fails, preserve the affected evidence and investigate the specific obligation. If secrets leak, stop accumulating affected data until the boundary is repaired. If a policy parameter is unresolved, use an explicitly versioned simulation policy and keep the live deployment gate closed.

## 24 Current Next Assignment

As agreed on 8 October 2026, proceed with Milestone 4: build the SwiftUI member application, shipping macOS first and establishing iOS targets from the outset. Keep business logic and portable client workflows in Rust, introduce the narrow Swift binding boundary, and validate member journeys locally with simulated payments and controlled email transports. Follow the Apple client architecture; the planned Tauri shell and desktop/web component-sharing approach are superseded. Do not provision or begin paying for the Linux host during this stage. Revisit hosting when the application is built and locally validated, ready for public HTTPS callbacks and incoming-email deployment checks.